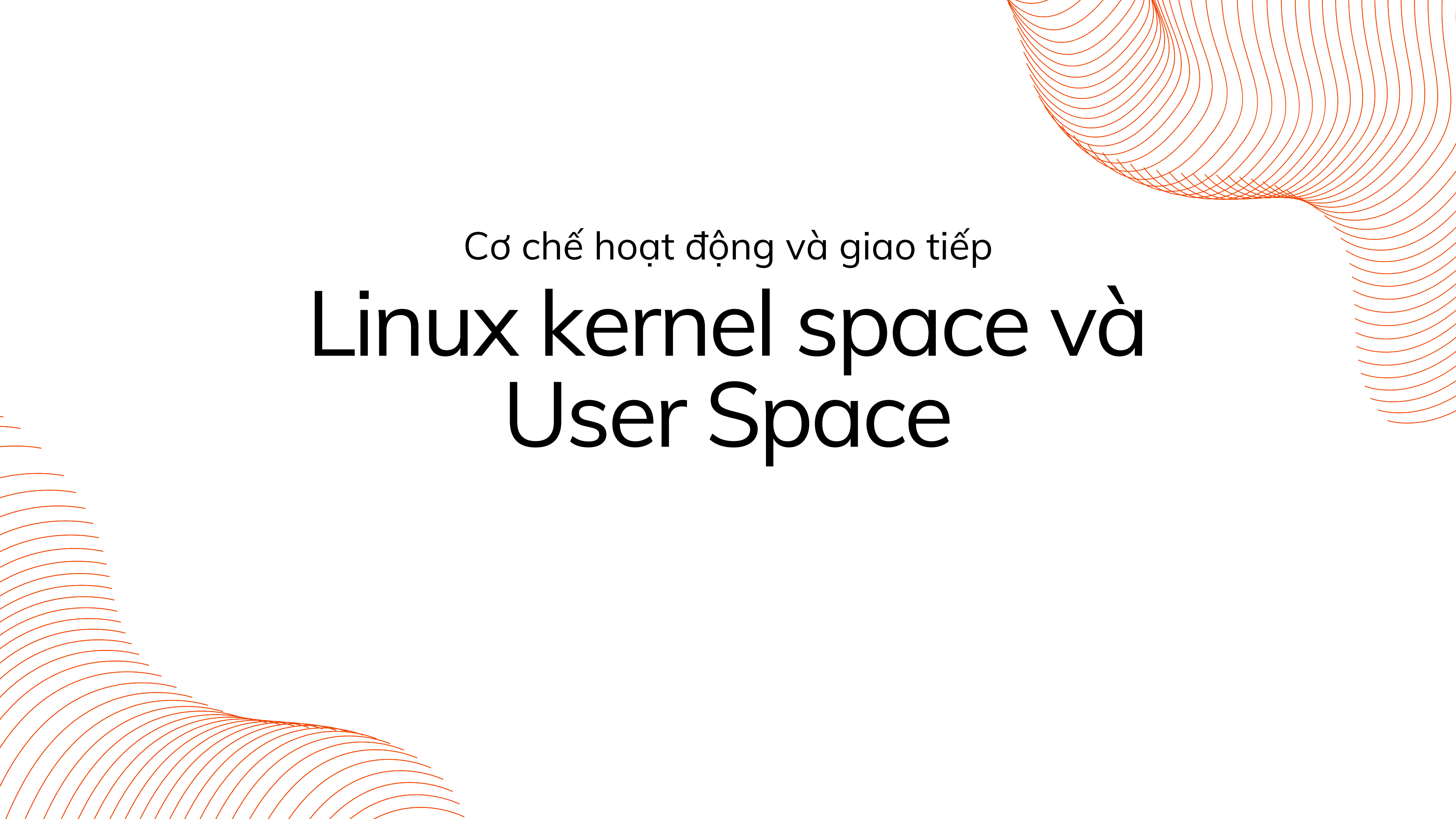




Cơ chế hoạt động và giao tiếp

Linux kernel space và User Space

Nội dung

- Cơ chế hoạt động của User Space
- Cơ chế hoạt động của Linux Kernel Space
- Phương thức giao tiếp giữa 2 Space
- Phương thức giao tiếp giữa 2 ứng dụng trong User Space



Cơ chế hoạt động

User Space và Linux Kernel Space

User Space

- Nơi các tiến trình hoạt động ở mức người dùng, thuộc Ring 3 trong Protection Ring
- Mỗi tiến trình đều có bộ nhớ ảo(virtual memory)=> tránh vi phạm cơ chế an toàn
- Truy cập phần cứng thông qua các lớp trừu tượng
- Chương trình cần sử dụng tài nguyên buộc phải thông qua sự cho phép và cấp phát của Kernel

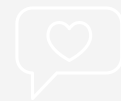


Linux Kernel Space

Quản lý tài nguyên và điều phối hoạt động của hệ điều hành

- Scheduler
- Memory Manager
- Process Management
- Network Handle
- System Call(*)

Có đặc quyền cao nhất



- Ring 0
- Truy cập trực tiếp vào phần cứng

Điều kiện kích hoạt



- System Call
- Interrupt
- Exception

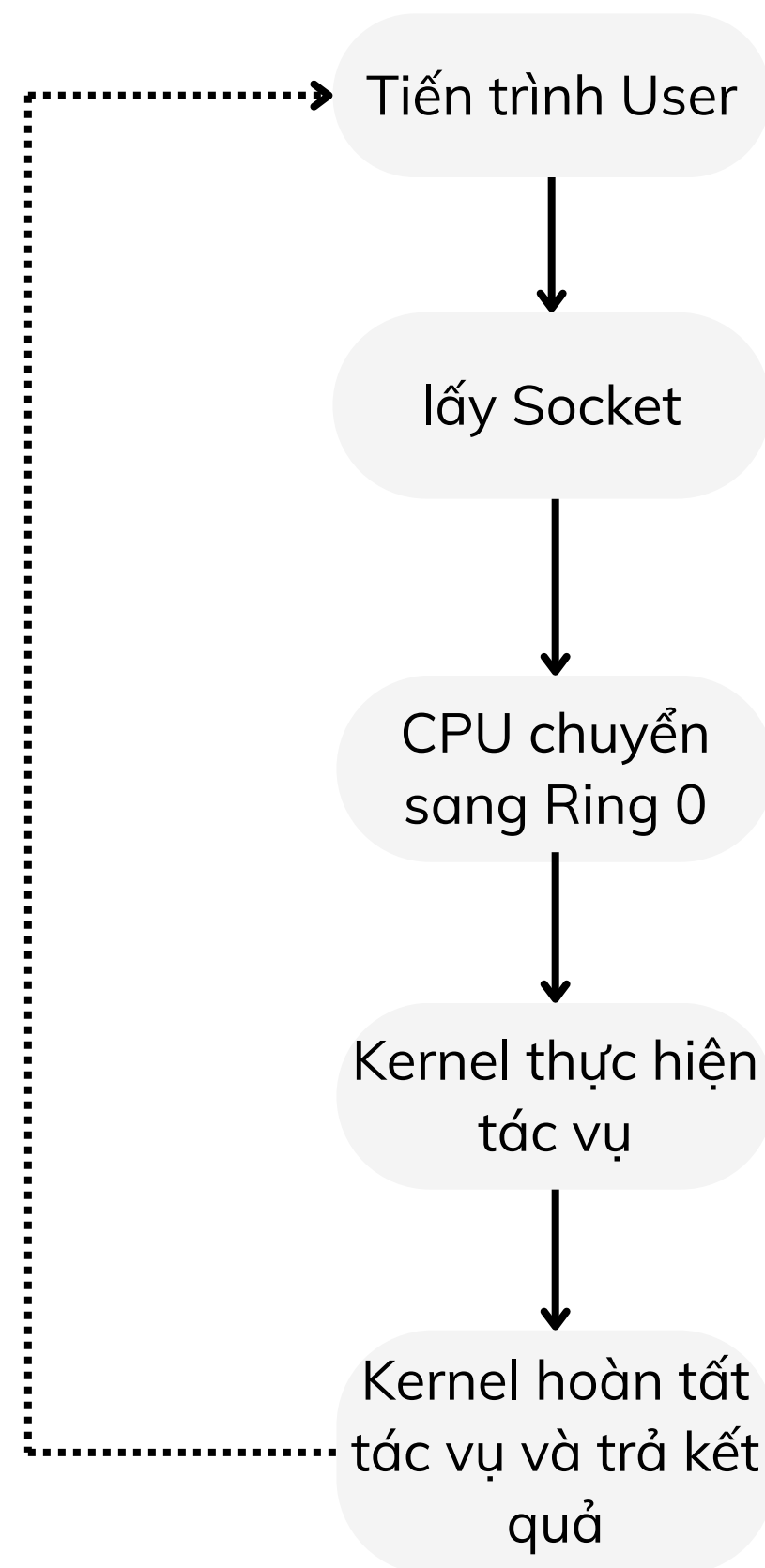
Cơ chế giao tiếp

User Space và Linux Kernel Space

System Call

Giao thức chính để tiến trình mức User Space giao tiếp với phần cứng thông qua Kenrel

Ví dụ: đọc file, tạo object, tạo kết nối với mạng, sử dụng thiết bị ngoại vi



Tiến trình thuộc Ring 3

Socket là cổng giao tiếp vào phần cứng

tiến trình User sẽ bị block hoặc trong trình trạng await

tác vụ sẽ được phép truy cập vào phần cứng của máy tính

Kernel hoàn thành tác vụ và trả lại quyền sử dụng cho tiến trình User và trở về mức Ring 3

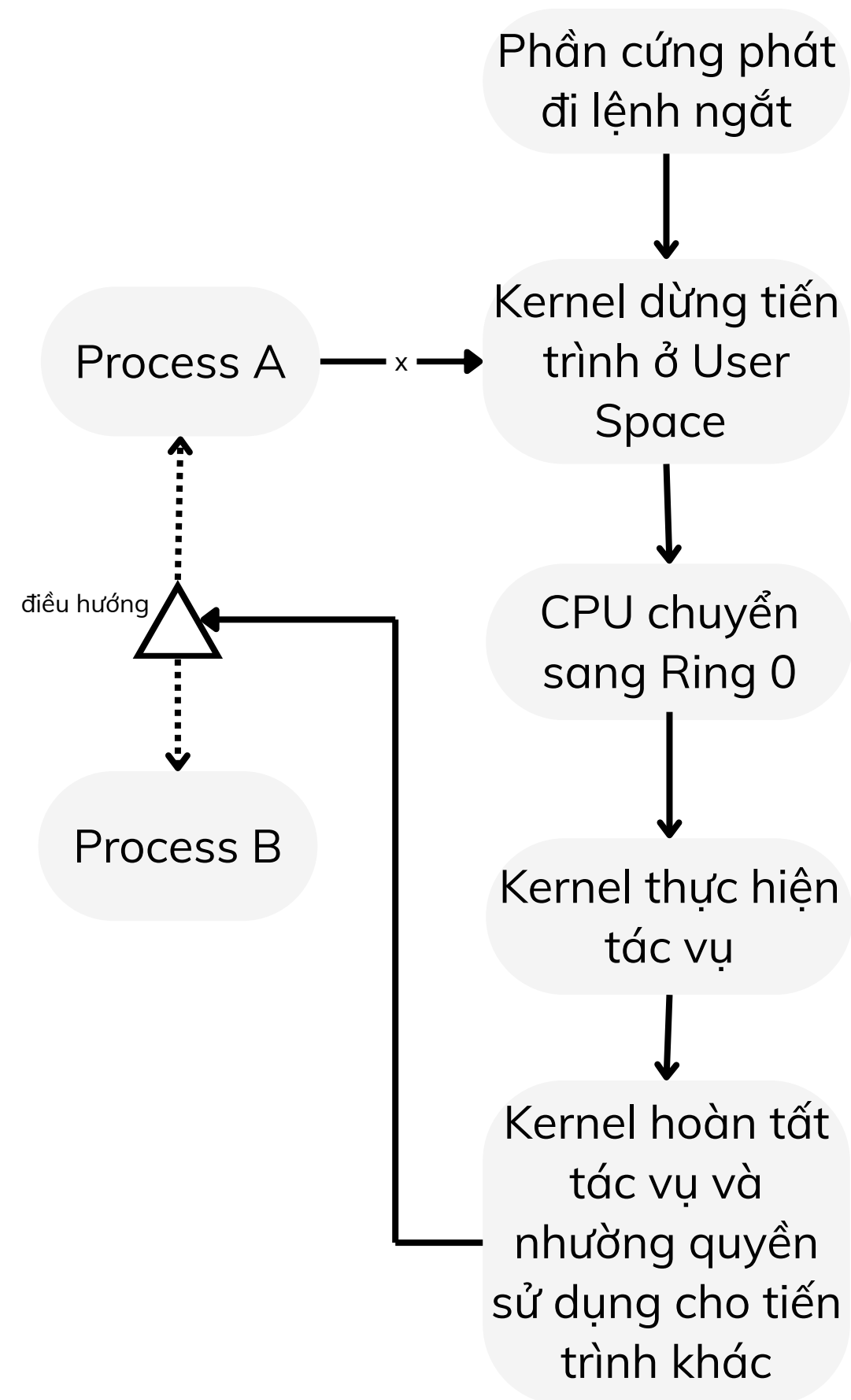
Interrupt

- Kernel sẽ ngắt tiến trình đang hoạt động ở CPU (có thể là tiến trình ở mức User) và sử dụng để xử lý tác vụ khác
- Sau khi xử lý xong, CPU có thể tiếp tục xử lý công việc trước đó hoặc tác vụ mà chúng ta đăng ký

=> Có thể sử dụng cơ chế làm cơ chế bất đồng bộ xử lý các tác vụ hiệu năng cao

Ví dụ: Cơ chế epoll, io_uring

Thư viện áp dụng cơ chế: Boost.Asio



Phần cứng nhận được tín hiệu từ bên ngoài hoặc nội bộ

Kernel yêu cầu tiến trình chuyển về chế độ block hoặc await

CPU được chuyển lên quyền cao nhất để xử lý với các tác vụ phần cứng

tác vụ sẽ được phép truy cập vào phần cứng của máy tính

Kernel hoàn thành tác vụ và trả lại quyền sử dụng cho tiến trình User và trở về mức Ring 3

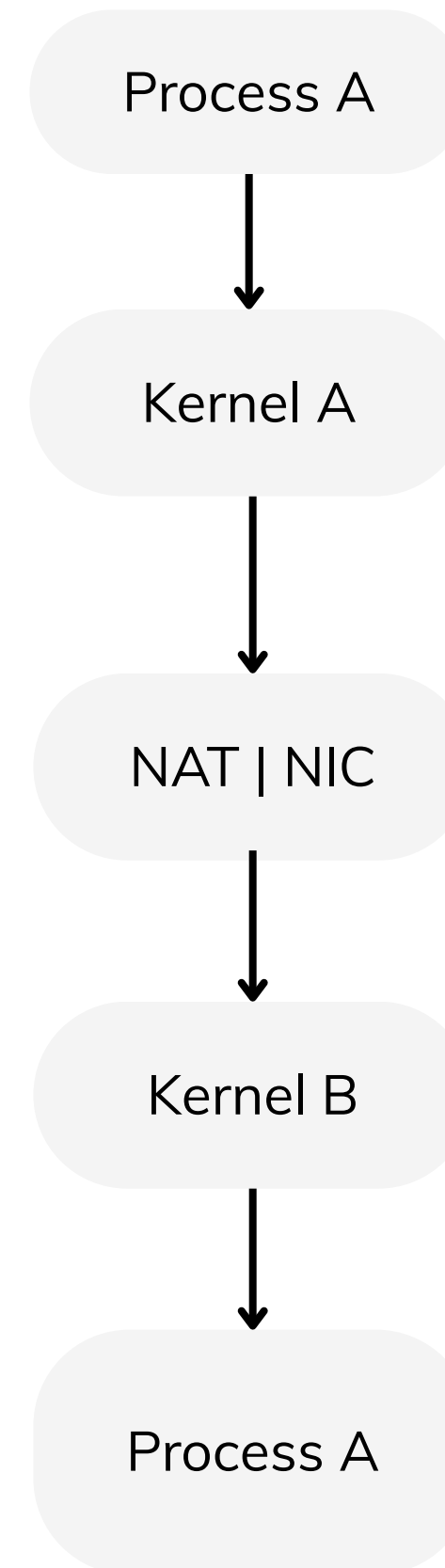
Cơ chế giao tiếp

2 tiến trình ở User Space

TCP/UDP Socket

Giao tiếp khi 2 chương trình không
cùng một mạng

Ví dụ: gửi dữ liệu tới các Server

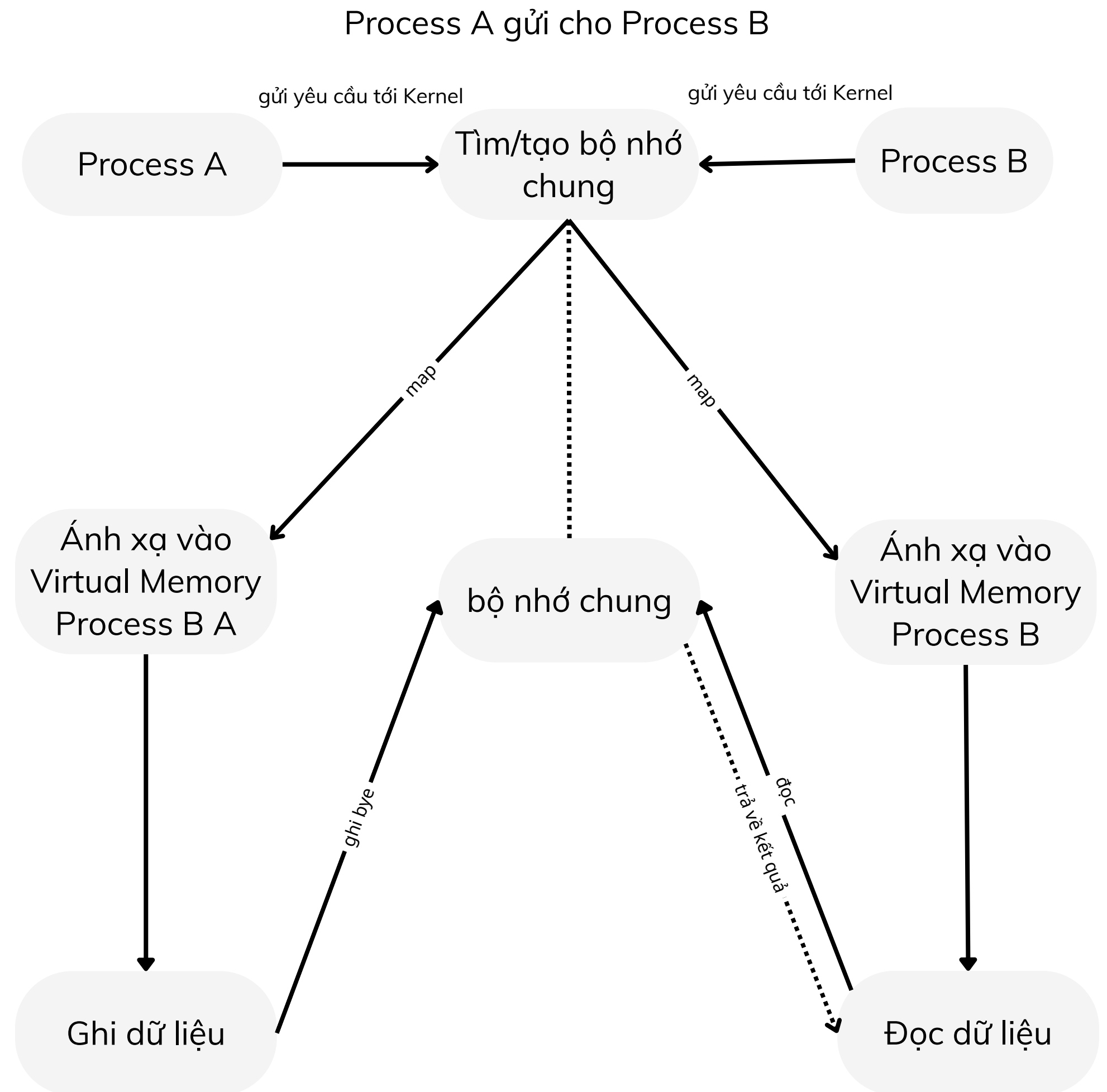


Nếu mạng thuộc chế độ NAT sẽ quay
lại về Kernel A, bằng không sẽ gửi
NIC và ra WAN

Shared Memory

- Tạo vùng nhớ chung thông qua Kernel để các tiến trình giao tiếp với nhau

=> Vì truy cập trên RAM nên có hiệu suất cao nhất



Message Queue

- Hàng đợi tin nhắn do Kernel quản lý
- Ý tưởng giống cách phân phát gói tin từ NIC tới Cache CPU

Khi nào nên dùng:

- Message có kiểu dữ liệu hoặc có sự ưu tiên
- Hàng đợi trung gian
- Producer và Consumer để phân phát tới các tiến trình

